Mobile Demo Script Explanation -- Filebrowser (Exploitation, Challenge 05)

Olawoyin Joseph Pamilerin, ISU520196

In one sentence: A leaked PendingIntent lets you execute a request as the target app; the request itself has a command-injection bug.

Background -- what the terms below mean

What does "Exploitation" mean, as a category? An **Exploitation** challenge gives you a real target Android app with an actual security mistake in it, and asks you to write a second, small app of your own that takes advantage of that mistake to get the flag.

What is a vulnerability? A mistake in how software was built that lets someone do something the developer never intended -- here, get the flag without going through the app the normal, intended way.

What is an APK, and what does "decompile" mean? An APK is the installable file for an Android app. It ships as compiled code that isn't directly readable, so we **decompile** it -- turn it back into readable, Java-like source code -- using a free tool called **jadx** (opened as **jadx-gui**) -- to find the mistake in the first place.

What is a package name? Every Android app has a unique ID string (like org.naija.fdexploit). Android refuses to install two different apps that both use the exact same ID, which occasionally trips up a first submission attempt if the exploit app accidentally reuses the target's own package name.

How the APK was obtained

The target APK was **downloaded directly from the challenge's own page** on challs.reyammer.io, before doing anything else. A copy of it is kept at:

```
C:\Users\Admin\Desktop\MOBISEC Challenge\Challenges\Exploitation/05 Filebrowser\Original Target APK.apk
```

How the solution was built

Built as a normal **Android Studio project** (Gradle). From a terminal: `cd` into its `Android Studio Project` folder and run `gradlew.bat assembleDebug`. The resulting APK is kept right there as **SUBMITTED APK Challenge 05 Filebrowser.apk**, ready to re-upload.

The vulnerability / technique, in plain terms

Two chained vulnerabilities: (1) a leaked PendingIntent from the exported PluginActivity actually targets the non-exported QueryActivity, letting us execute AS the filebrowser app by filling in its extras; (2) the 'arg' extra is concatenated unsanitized into a shell command, allowing OS command injection. We grep the AES key from shared_prefs, then grep+decrypt the flag out of the raw SQLite DB.

Files to open

1. Open the target APK in jadx-gui:

```
C:\Users\Admin\Desktop\MOBISEC Challenge\Challenges\Exploitation/05 Filebrowser\Original Target APK.apk
```

2. MainActivity.java:

```
C:\Users\Admin\Desktop\MOBISEC Challenge\Challenges\Exploitation/05 Filebrowser\Android Studio Project\app\s
```

This is our own code -- open directly in Android Studio or any text editor.

What to say -- explanation of the exploit, step by step

1. I decompiled the target and found a PendingIntent leaked from its exported PluginActivity actually targets its non-exported QueryActivity.

2. I fired that PendingIntent with my own extras filled in, letting me execute a query as the filebrowser app itself.

3. I discovered the query's "arg" extra is concatenated unsanitized into a shell command, so I used it for OS command injection to read the app's shared_prefs and grep out its AES key.

4. I used that key to decrypt the raw SQLite database directly and recover the flag.

5. I built and submitted the APK; the platform's analysis log confirmed each stage worked and released the flag.

If an emulator isn't available

If asked how you got/confirmed the flag: "I built my exploit app and uploaded the APK to the platform's own submission page. Their server ran it automatically against the target and returned the flag in the analysis log." This is backed by a real screenshot of that exact submission page (see **Real Screenshot: Live Platform Submission Page** in this challenge's Report.pdf), not just a claim -- worth pointing to directly if an emulator isn't running.

Result

The flag this challenge produced:

```
MOBISEC{a_good_file_browser_would_gimme_the_flag_with_no_hacks}
```

Source: Verbatim from /apk/81693 analysis log

The real, verbatim output this run produced:

```
got PendingIntent, stage 1: reading key
recovered key: YEpEhlLnsUynUHunNKFI
stage 2: grepping raw db for oper=97a4588a91d4cb28957210b9933cbe0b
FLAG: MOBISEC{a_good_file_browser_would_gimme_the_flag_with_no_hacks}
```